MEDITOKEN - COMPLETE VIVA & PROJECT GUIDE

Doctor Appointment Booking & Intelligent Digital Token Queue System

Project Title: MediToken - Digital Healthcare Queue & Appointment System
Academic Level: Final Year IT / CS Capstone Project | **Type:** Full-Stack Web Application
Tech Stack: PHP 8.2 (Backend), MySQL / MariaDB (Database), HTML5 / Vanilla CSS3 / JavaScript AJAX (Frontend)
Deployment: Docker + Apache on Railway Cloud (Live: meditoken.up.railway.app) & Localhost (XAMPP)

1. Project Introduction & Problem Statement (Kyu Banaya?)

Real Problem: Traditional OPDs aur Clinics me sabse badi samasya manual paper tokens aur lambi bheed hai. Patients ko pata nahi hota unka number kab aayega, physical paper slips kho jaati hain, aur overcrowding se hospital management chaotic ho jata hai.
MediToken Solution: MediToken ek **100% database-driven web application** hai. Yeh patient ko digital token (jaise A-001) deta hai, double-booking prevent karta hai, aur live AJAX polling se mobile par hi real-time queue status (abhi kiska checkup chal raha hai, kitne log aage hain, expected wait time) show karta hai.

2. System Architecture & 3 User Portals (Kaun Kya Karta Hai?)

Portal / User	Main Role & Purpose	Key Features & Actions
1. Patient (Mareez)	Doctor search karna, time-slot book karna, aur live queue monitor karna.	• Registration & Secure Login • Book Appointment (Doctor, Date, Slot) • Instant Digital Token (A-001) • Real-time Live Queue Tracker (Mera Number Kab Aayega) • Cancel Appointment & In-App Alerts
2. Doctor (Chikitsak)	Apne cabin me OPD queue ko single-click se run aur control karna.	• Doctor Login Desk • Live OPD Queue Dashboard • 'Call Next Token' (Agle patient ko Serving me lana) • 'Complete Consultation' (Checkup khatam mark karna) • 'Skip / No-Show' (Agar patient present na ho) • Patient History & Daily Count
3. Admin (Hospital Admin)	Overall clinic governance, doctor management, schedules, aur audit reports.	• Master Administrator Dashboard • Doctor CRUD (Add, Edit, Specialization, Timings) • Doctor Availability Toggle (Available/Unavailable) • Multi-Doctor Live Queue Monitoring • Patient Registry & Daily Analytics Reports

3. Relational Database Design (meditoken_db - 5 Core Tables)

Table Name	Key Columns & Purpose	Relationships & Indexes
patients	`patient_id` (PK), `name`, `mobile_no`, `email` (UNIQUE), `password` (Bcrypt hash), `created_at`	1 Patient -> Multiple Appointments
doctors	`doctor_id` (PK), `doctor_name`, `email` (UNIQUE), `specialization`, `available_time`, `availability_status`	1 Doctor -> Multiple Appointments
appointments *(Central Table)*	`appointment_id` (PK), `patient_id` (FK), `doctor_id` (FK), `appointment_date`, `appointment_time`, `token_number` (A-001), `status` (Waiting, Serving, Completed, Cancelled)	Junction Table. UNIQUE `(doctor_id, appointment_date, token_number)` ensures zero duplicate tokens.
admins	`admin_id` (PK), `username` (UNIQUE), `password` (Bcrypt), `created_at`	Clinic Master Administration
notifications	`notification_id` (PK), `patient_id` (FK), `doctor_id`, `message`, `type`, `is_read`	Real-time Queue & Status Notifications

4. Core Technical Highlights & Algorithm Logic

• **Algorithmic Token Generation:** Random string nahi hoti. Doctor ke naam ke first alphabet ka prefix (jaise Dr. Amit -> 'A') aur us din ka maximum token query karke increment hota hai: `A-001`, `A-002`, `A-003`. Composite UNIQUE key lagne ki wajah se race condition me bhi duplicate token generate nahi ho sakta.
• **Real-Time Live Queue Polling (AJAX):** Heavy WebSockets ke badle lightweight **AJAX Long Polling (Fetch API)** use kiya gaya hai jo har 4 seconds me `api/queue_status.php` ko query karta hai. Screen bina flicker hue 'Now Serving' aur 'Waiting Count' live change hota hai.
• **Appointment Lifecycle State Machine:** Status flow: **[Waiting]** ➔ *(Doctor calls)* ➔ **[Serving]** ➔ *(Checkup complete)* ➔ **[Completed]**. Patient checkup se pehle cancel kar sakta hai ➔ **[Cancelled]**.
• **Security Architecture:** SQL Injection se bachne ke liye **PDO Prepared Statements** (`prepare` + `execute([$param])`), password security ke liye **Bcrypt (`PASSWORD_BCRYPT`)**, XSS protection ke liye `htmlspecialchars()`, aur role-based session authorization.

5. Top 15 Viva Questions & Winning Answers (Examiner FAQs)

Viva me External Examiner ke dwara pooche jane wale sabse zaroori questions aur unke best answers:

Q1. What is MediToken and what problem does it solve?

Ans: MediToken is a full-stack, database-driven digital queue and appointment booking system. It replaces physical OPD paper slips with automated digital tokens and allows patients to track their live queue position remotely, preventing crowding and long waiting times in clinics.

Q2. What is the Technology Stack used in MediToken?

Ans: Frontend: HTML5 Semantic tags, Vanilla CSS3 (custom medical design system without heavy frameworks), Vanilla JavaScript (ES6+ AJAX Fetch). Backend: PHP 8.2 (Modular Architecture, Session Auth, PDO). Database: MySQL / MariaDB (3NF relational schema). Deployment: Docker container on Railway Cloud + Localhost on XAMPP.

Q3. How is the Token Number generated? Can duplicates happen?

Ans: Tokens are generated algorithmically per doctor per date. The system takes the doctor's initial letter as prefix and queries the maximum assigned token for that day (`SELECT MAX(token_number)`), incrementing it by 1 (e.g. A-001, A-002). Duplicates are mathematically impossible because the MySQL database enforces a composite UNIQUE index `(doctor_id, appointment_date, token_number)`.

Q4. How does the live queue update in real-time without refreshing the page?

Ans: We implemented AJAX polling using the modern JavaScript `fetch()` API. Every 4 seconds, the client sends an asynchronous background request to `api/queue_status.php`. The backend queries the current 'Serving' token and remaining 'Waiting' patients, returning JSON. The DOM dynamically updates without page reloads.

Q5. How did you secure the application against SQL Injection?

Ans: We used PHP Data Objects (PDO) with parameterized prepared statements (`$stmt->prepare()` and `$stmt->execute([$params])`). Furthermore, `PDO::ATTR_EMULATE_PREPARES` is disabled so SQL statements are compiled on the MySQL database engine before any parameters are passed, completely preventing SQL Injection.

Q6. How are passwords stored and authenticated?

Ans: Passwords are never stored in plain text. They are hashed using the industry-standard Bcrypt cryptographic algorithm via PHP's native `password_hash($password, PASSWORD_BCRYPT)` with automatic salting. During login, `password_verify()` authenticates the hash securely.

Q7. How do you prevent double-booking for the exact same slot?

Ans: During booking, the system executes a concurrency pre-check query: `SELECT COUNT(*) FROM appointments WHERE doctor_id=? AND appointment_date=? AND appointment_time=? AND status != 'Cancelled'`. If an active appointment already exists for that slot, the transaction is rejected and the user is prompted to pick another slot.

Q8. What happens when an appointment is cancelled?

Ans: The appointment status is updated to 'Cancelled' in the database. The system calls `add_notification()` to log an alert for the patient and doctor. The live queue immediately recalculates the waiting count and updates the estimated wait time for subsequent patients in line.

Q9. What are the appointment lifecycle states?

Ans: There are 4 main states: **Waiting** (patient has booked token and is in queue) ➔ **Serving** (doctor called the token into cabin) ➔ **Completed** (consultation finished) or **Cancelled** (appointment cancelled before consultation).

Q10. How is Role-Based Access Control (RBAC) implemented?

Ans: PHP session variables (`$_SESSION['user_id']`, `$_SESSION['role']`) are set upon login. Every restricted page invokes `require_role('admin')`, `require_role('doctor')`, or `require_role('patient')`. If an unauthorized role or guest tries to access a protected URL, they are immediately redirected to the login page.

Q11. Why did you choose Vanilla CSS instead of Bootstrap or Tailwind?

Ans: To create a clean, high-performance, and lightweight medical theme with zero external bloat, ensuring maximum page loading speed, full control over responsive Flexbox/Grid layouts, and excellent high-contrast readability across mobile and desktop screens.

Q12. What are the key tables and relationships in your database?

Ans: The database has 5 tables in 3NF: `patients` (patient credentials), `doctors` (specializations and schedules), `appointments` (junction table connecting patients and doctors with token & status), `admins` (administrative credentials), and `notifications` (queue event alerts). Foreign keys use `ON DELETE CASCADE` to maintain referential integrity.

Q13. How did you deploy the application to Railway Cloud?

Ans: We containerized the app using a Dockerfile based on `php:8.2-apache` with `pdo_mysql` and `mpm_prefork`. A custom `docker-entrypoint.sh` binds Apache to Railway's dynamic `$PORT`. Database environment variables (`MYSQLHOST`, `MYSQLPORT`, `MYSQLUSER`, `MYSQLPASSWORD`, `MYSQLDATABASE`) automatically connect the app to Railway's managed MySQL database.

Q14. How does the Doctor desk operate?

Ans: The doctor logs in and opens the 'Live Queue Desk'. It shows today's queued patients sorted by token number. With one click on 'Call Next', the highest-priority waiting token transitions to 'Serving'. After the checkup, clicking 'Complete' marks it done and allows calling the next patient.

Q15. What are the future enhancements for MediToken?

Ans: Future scope includes: 1) SMS/WhatsApp API (Twilio/Fast2SMS) for automated alerts when the patient's turn is 2 tokens away, 2) Payment Gateway integration (Razorpay/Stripe) for online consultation fees, and 3) An AI-based wait time predictor that estimates delays based on average consultation durations.